

VI. Aprendizaje Basado en Casos

Aprendizaje Automático - Instituto de Computación - UdelaR

Índice

- Introducción
- K-Nearest Neighbor
- Regresión local ponderada
- Razonamiento basado en casos
- Algoritmos perezosos vs. algoritmos ansiosos

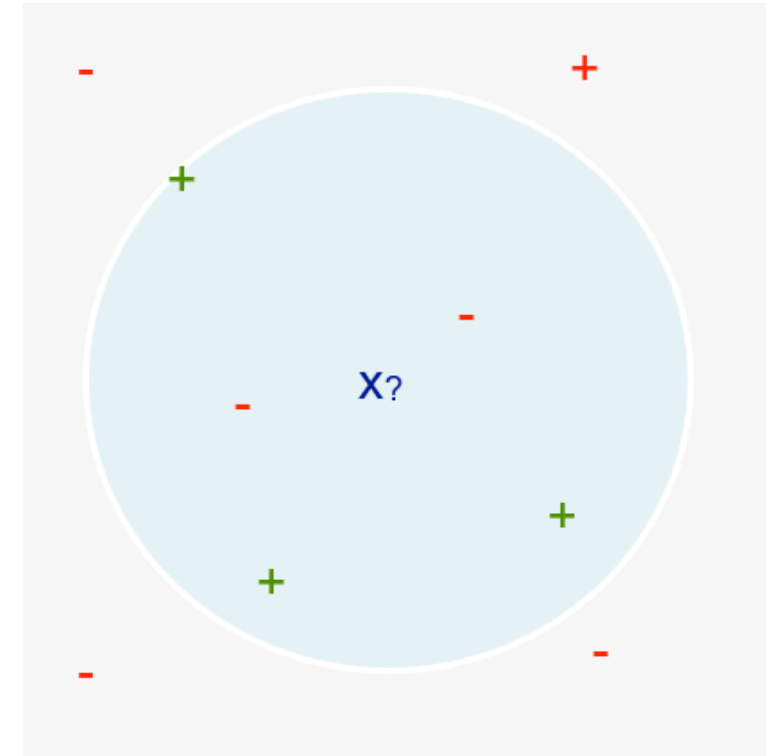
Introducción

- La idea es crear un clasificador 'perezoso'.
- Para clasificar una nueva instancia, utilizo aquellas que más se le parecen de las que ya conozco.
- Ventajas:
 - Para cada nueva instancia puedo obtener un clasificador diferente.
 - La descripción de la instancia puede ser tan compleja como quiera.
- Desventajas:
 - El costo de clasificación puede ser alto.
 - Atributos irrelevantes pueden afectar la medida de similitud.

K-Nearest Neighbor

- Queremos aproximar un concepto utilizando las k instancias más cercanas a un elemento $\langle a_1, \dots, a_n \rangle$ que deseamos clasificar.
- Utilizamos la función de distancia euclidiana:

$$d(\langle a_1, \dots, a_n \rangle, \langle b_1, \dots, b_n \rangle) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$



K-Nearest Neighbor

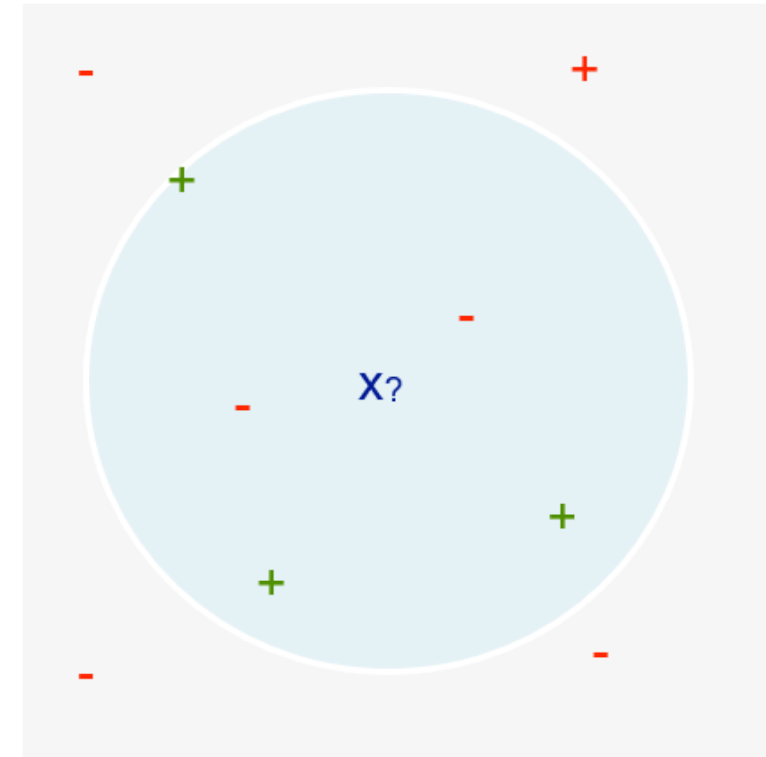
- El valor que asignamos a $x_?$ es:

$$h(x_?) \leftarrow \arg \max_{v \in \{+, -\}} \sum_{x_i \in \text{k-nn}(x_?) } \delta(v, f(x_i))$$

(siendo $\delta(a, b) = 1$ si $a = b$ y $\delta(a, b) = 0$ en otro caso)

- Si la función es real, se puede utilizar:

$$h(x_?) \leftarrow \frac{1}{k} \sum_{x_i \in \text{k-nn}(x_?) } f(x_i)$$



K-Nearest Neighbor

- Los ejemplos más cercanos a la instancia $x_?$ podrían considerarse más importantes que los lejanos...

$$h_{dis}(x_?) \leftarrow \operatorname{argmax}_{v \in \{+, -\}} \sum_{x_i \in \text{k-nn}(x_?)} w_i \cdot \delta(v, f(x_i))$$

$$h_{real}(x_?) \leftarrow \sum_{x_i \in \text{k-nn}(x_?)} \frac{w_i \cdot f(x_i)}{\sum w_i}$$

- Con $w_i = \frac{1}{d(x_i, x_?)^2}$
- Si ponderamos por la distancia, podemos usar a todo el conjunto para evaluar una nueva instancia. [Método de Shepard]

K-Nearest Neighbor

- Al tomar en cuenta a varios vecinos, el algoritmo es menos sensible al ruido.
- Pero ¿cuántos vecinos elegir?
 - Si se elige k muy bajo, el resultado es muy sensible al ruido; si es muy alto, las zonas que tengan muchos ejemplos pueden acaparar a zonas que tengan menos.
 - Por lo general se elige un k impar para no tener problemas de empate. Los valores usuales son bajos: 1, 3 y 5.
 - Una forma de estimar k es probando distintos valores, midiendo la performance dejando un elemento del conjunto afuera y clasificando con el resto (1-out-cross-validation).

K-Nearest Neighbor

- A diferencia de los árboles de decisión, al aplicar la distancia euclidiana, se tienen en cuenta TODAS las dimensiones que describen un atributo
- Problemas
 - si 2 atributos en 20 son los relevantes, instancias que en realidad son muy diferentes pueden estar muy próximas en el espacio (maldición de las dimensiones).
 - ¿Qué sucede con atributos con distintas magnitudes?

K-Nearest Neighbor

- Maldición de las dimensiones:
 - Una posible solución es asignar pesos a los distintos atributos. Esto corresponde a modificar el largo de los ejes del espacio.
 - ¿Cómo se determina cuánto hay que acortar o alargar un eje?
 - Penalizando los atributos cuya distribución es uniforme en cada clase.
 - Utilizando validación cruzada.

K-Nearest Neighbor

- ¿Qué sucede con atributos con distintas magnitudes?
 - Estandarización: se asume que A es $\mathcal{N}(\mu_A, \sigma_A)$ y se lleva a $\mathcal{N}(0, 1)$

$$Est(X, A) = \frac{x - \mu_A}{\sigma_A}$$

- Reescalamiento: se cambia el rango de los atributos

$$Resc(x, A) = \frac{x - \min_A}{\max_A - \min_A}$$

K-Nearest Neighbor

- Cada clasificación implica la selección de k ejemplos; esto puede ser muy costoso si contamos con un gran conjunto de ejemplos.
- Esto se soluciona implementando una buena indexación sobre el conjunto.
- ¿Cuál es el sesgo inductivo de k -NN?

La clasificación de una instancia es parecida a las de sus k vecinos (cercanía implica similitud).

Regresión Local Ponderada

- Generalizamos KNN, creando una aproximación local de la función objetivo y construimos una función h que aproxime a los puntos cercanos a x ?
- Podemos elegir qué modelo de función utilizamos para esta aproximación (lineal, cuadrática, etc.). Por ejemplo:

$$h(\langle a_1, \dots, a_n \rangle) = w_0 + w_1 \cdot a_1 + \dots + w_n \cdot a_n$$

... y buscamos el vector $\langle w_0, w_1, \dots, w_n \rangle$ que minimice el error de la función sobre los k puntos más cercanos (veremos en mucho más detalle esto en las clases de regresión, regresión logística y redes neuronales)

- La pregunta es: ¿qué función de error deberíamos considerar?

Regresión Local Ponderada

- Sobre los k vecinos más cercanos:

$$E = \frac{1}{2} \sum_{x \in k_n n(x_?) } (f(x) - h(x))^2$$

- Ponderando a todo el conjunto:

$$E = \frac{1}{2} \sum_{x \in D} (f(x) - h(x))^2 K(d(x, x_?))$$

- Ponderando a los k vecinos más cercanos

$$E = \frac{1}{2} \sum_{x \in k_n n(x_?) } (f(x) - h(x))^2 K(d(x, x_?))$$

(K es una función decreciente)

Regresión Local Ponderada

- En la selección de la función de error se debe considerar cuánto queremos que influya el total del conjunto de entrenamiento vs. el costo computacional.
- Debido al costo en la reducción del error, se utilizan por lo general funciones lineales o cuadráticas.
- Luego de clasificar la nueva instancia, podemos descartar la hipótesis encontrada: cada instancia genera una nueva aproximación.

Razonamiento basado en casos

- ¿Qué sucede cuando las instancias son representadas de forma más compleja?

Transporte: $\langle \text{omnibus} \rangle$

Tiempo: $\langle \text{dia}_{+1}:\text{nublado}, \text{dia}_{+2}:\text{soleado}, \text{dia}_{+3}:? \rangle$

Lugar: $\langle \text{casa}[\text{cuartos}:2, \text{piscina}:\text{no}], \text{centro}:10 \text{ km} \rangle$

Personas: $\langle \text{adultos}(\text{hombres}: 1, \text{mujeres}: 1), \text{niños}=4 \rangle$

Satisfacción: ???

- Al igual que KNN y RLP, clasificamos una instancia en base a casos parecidos: en lugar de puntos en un espacio euclídeo, las instancias se representan con atributos más complejos. Se debe buscar una métrica de similitud que depende del dominio de trabajo. La solución se basa en combinaciones complejas y específicas al dominio de aplicación.

Algoritmos perezosos versus ansiosos

- Los algoritmos que vimos (KNN, Regresión Local...) difieren el cálculo de una hipótesis hasta la llegada de una nueva consulta (algoritmos perezosos).
- Estos algoritmos computan una aproximación local de la función objetivo para responder cada nueva consulta: utilizan múltiples aproximaciones locales para modelar la función objetivo [global].
- Los algoritmos ansiosos pueden utilizar también aproximaciones; sin embargo, éstas quedan «fijas» al conjunto de entrenamiento.

Algoritmos perezosos versus ansiosos

- Dado un mismo espacio de hipótesis, los algoritmos perezosos tienen una mayor poder de adaptación a una nueva consulta.
- El costo de clasificación, sin embargo, aumenta: la aproximación se realiza en tiempo de clasificación y no de entrenamiento.
- Se precisan estructuras eficientes para determinar los ejemplos cercanos a la instancia a clasificar.

¿Preguntas?

- Bibliografía: "Machine Learning" (Tom Mitchell). Capítulo 8.